

Superseded Event-Based Design Notes

Working Document

September 8, 2026

Status: superseded. This document describes an earlier successor-arc event formulation and is retained for historical context. The authoritative event formulation is the ordered E1–E15 model in `paper/sections/06_event_reformulation.tex`; its paper-facing implementation is `OrderedAEventMILPScheduler`.

1 Modeling Principle

The original maintenance model repeats the planning-day index in several variables even though each flight already has an absolute departure and arrival timestamp. If all times are measured from the beginning of the planning horizon, the calendar day of flight i is derived data, $\lfloor t_i^{\text{dep}}/1440 \rfloor$, rather than a decision index.

Maintenance is therefore represented as a virtual flight from an airport to the same airport. It starts immediately after a scheduled flight, has a fixed duration, occupies the aircraft, and consumes station capacity. The new binary variable z_{ijk} selects maintenance type k after flight i for aircraft j . This removes the day dimension from the maintenance variables while preserving all calendar information in the timestamps.

2 Update Summary for C1–C14

The C1–C14 update is based on one modeling decision: *do not use planning days as decision indices when absolute flight times already carry the same calendar information.*

The resulting formulation is:

1. Keep flight assignment and route sequencing in event time.
2. Represent each maintenance action as a virtual self-loop leg $a \rightarrow a$ with fixed duration δ_k .
3. Use the maintenance variable

$$z_{ijk} = 1 \iff \text{aircraft } j \text{ performs maintenance type } k \text{ after flight } i.$$

Thus, maintenance is attached directly to flight events, not to day buckets. This removes day-indexed binaries while keeping timing, capacity, and maintenance-feasibility logic fully time-consistent.

3 A-only ordered-state prototype

As a compactness experiment, an A-only prototype orders all flights globally by departure time and replaces the explicit successor variable $w_{i\ell j}$ with one prefix state q_{rj} for each ordered flight position r and aircraft j . This construction is valid only because the A-only state can be updated while scanning the total time order; route continuity remains enforced by the corrected basic routing constraints. If flight i_r is not assigned to j , the state is carried forward. If it is assigned without an A check, its duration is added. If $z_{i_r j A} = 1$, the state is reset after that flight.

Let $x_{i_r j}$ be the assignment variable, $z_{i_r j A}$ the A-check event, and p_{i_r} the flight duration. With $q_{-1,j} = \Phi_{jA}$, a linear big- M implementation uses:

$$q_{rj} \geq q_{r-1,j} - M_{rj}x_{i_r j}, \quad (1)$$

$$q_{rj} \leq q_{r-1,j} + M_{rj}x_{i_r j}, \quad (2)$$

$$q_{rj} \geq q_{r-1,j} + p_{i_r} - M_{rj}(1 - x_{i_r j}) - M_{rj}z_{i_r j A}, \quad (3)$$

$$q_{rj} \leq q_{r-1,j} + p_{i_r} + M_{rj}(1 - x_{i_r j}) + M_{rj}z_{i_r j A}, \quad (4)$$

$$q_{rj} \leq p_{i_r}z_{i_r j A} + T_A(x_{i_r j} - z_{i_r j A}) + M_{rj}(1 - x_{i_r j}), \quad (5)$$

where $M_{rj} = \max\{T_A, \Phi_{jA}\} + p_{i_r}$. The state is bounded by $0 \leq q_{rj} \leq \max\{T_A, \Phi_{jA}\}$, and every assigned flight is required to satisfy $q_{rj} \leq T_A$. In addition, every assigned flight must satisfy $q_{r-1,j} + p_{i_r} \leq T_A$ before a post-flight check can reset the state; a check after an overdue flight cannot repair the violation. The first two rows carry the state when the flight is not assigned; the next two rows impose accumulation or relax the transition at a reset; the final row gives $q_{rj} = p_{i_r}$ at a selected reset and $q_{rj} \leq T_A$ for an assigned non-reset flight.

This ordered-state construction removes $w_{i\ell j}$ only for the A-only prototype. It must not be generalized to the full hierarchy without adding separate timestamped C/D event state and proving that route continuity and maintenance timing remain equivalent. In the current implementation it is provided as `OrderedAEventMILPScheduler` for controlled parity experiments; the full validated formulation remains `EventMILPScheduler`.

The same ordered prefix-state construction has been extended to the hour-based A/B subset as `OrderedABEventMILPScheduler`. It maintains one q_{rjA} and one q_{rjB} state for each ordered flight position and aircraft. An A check resets only the A state, while a B check resets both states through the hierarchy relation. On the clean and near-threshold A/B test instances, the prototype returns the same optimal objective as the legacy A/B model

while using fewer variables. This is an A/B validation milestone, not yet a claim for the C/D calendar hierarchy.

The full ordered A/B/C/D prototype has also been tested on the available feasible ABCD family. On the $h=7$ instance, the legacy, ordered, and standalone event models all return the optimal objective 10 327. The $h=15$, $h=21$, and $h=30$ files in that family have the same small flight set and all three models report infeasibility, so they are not treated as horizon-scaling evidence. The ordered model is therefore ready for broader ABCD testing, but its full performance claim remains provisional until larger feasible calendar-check instances are generated.

As a separate fleet-size experiment, one-rotation feasible ABCD instances were generated for $|\mathcal{P}| \in \{5, 10, 15, 20\}$, producing 10, 20, 30, and 40 flights. The reproducible comparison is implemented in `experiments/compare_ordered_hierarchy.py` and writes `results/tables/formulation_abcd_fleet_scaling_repro.csv`. Legacy, ordered, and standalone event models are optimal with identical objectives at all four fleet sizes. The ordered model uses 47–72% fewer variables than legacy; its constraint count is lower for $|\mathcal{P}| = 5, 10$, approximately comparable at 15, and higher at 20. CPU times are close on this small grid, so these results support model-size reduction and objective parity, not a universal speedup.

Index choice for maintenance variables. The robust event-linked variable is z_{ijk} : check type k after flight i for aircraft j . A reduced variable such as z_{jk} is not sufficient in the integrated routing model, because the model must know *after which flight* the check occurs to enforce turn-time blocking, station capacity over time, and hour/calendar reset logic. If one wants to remove flight index i , an explicit event-position index (for example, route position r) is still required, which reintroduces sequencing complexity elsewhere.

4 Sets and Parameters

Let $\mathcal{F}$ be the scheduled flights, $\mathcal{P}$ the aircraft, $\mathcal{A}$ the airports, $\mathcal{M} \subseteq \mathcal{A}$ the maintenance-capable airports, and $\mathcal{C} = \{A, B, C, D\}$ the check types. Indices i and ℓ denote flights, j denotes an aircraft, and k denotes a check type. Define $\mathcal{C}_H = \{A, B\}$ for flight-hour checks and $\mathcal{C}_T = \{C, D\}$ for calendar-time checks.

For flight i , let o_i , a_i , t_i^{dep} , t_i^{arr} , and $p_i = t_i^{\text{arr}} - t_i^{\text{dep}}$ be its origin, destination, departure time, arrival time, and flying time. For aircraft j , let a_j^0 be its initial airport and r_j its initial availability time. Let Δ be the minimum turn time and δ_k the fixed duration of check k . Maintenance attached to flight i starts at $\sigma_i = t_i^{\text{arr}}$ and is the virtual leg

$$m(i, j, k) = (a_i, a_i, \sigma_i, \sigma_i + \delta_k). \quad (6)$$

Only flights in $\mathcal{F}^M = \{i \in \mathcal{F} : a_i \in \mathcal{M}\}$ may be followed by maintenance. The sparse connection set is

$$\mathcal{E} = \left\{ (i, \ell) \in \mathcal{F} \times \mathcal{F} : a_i = o_\ell, \quad t_\ell^{\text{dep}} \geq t_i^{\text{arr}} + \Delta \right\}. \quad (7)$$

For each check type, define the connections that leave enough time for that maintenance event:

$$\mathcal{E}_k = \left\{ (i, \ell) \in \mathcal{E} : t_\ell^{\text{dep}} \geq \sigma_i + \delta_k + \Delta \right\}. \quad (8)$$

The aircraft-specific set of feasible first flights is

$$\mathcal{F}_j^0 = \left\{ i \in \mathcal{F} : o_i = a_j^0, \quad t_i^{\text{dep}} \geq r_j \right\}. \quad (9)$$

Let T_k be the maximum flying time between qualifying checks for $k \in \mathcal{C}_H$, and let L_k be the maximum elapsed time between qualifying checks for $k \in \mathcal{C}_T$. A limit specified as d_k calendar days is stored as $L_k = 1440d_k$ minutes. Parameters Φ_{jk} and Ψ_{jk} are the pre-horizon accumulated flying time and elapsed calendar time, respectively. The relation $k' \succeq k$ means that check k' is at least as heavy as k .

5 Decision Variables and Objective

$$x_{ij} \in \{0, 1\}, \quad \text{flight } i \text{ is assigned to aircraft } j, \quad (10)$$

$$y_{ilj} \in \{0, 1\}, \quad \text{aircraft } j \text{ operates } \ell \text{ immediately after } i, \quad (11)$$

$$z_{ijk} \in \{0, 1\}, \quad \text{aircraft } j \text{ receives check } k \text{ immediately after } i, \quad (12)$$

$$u_{ijk} \geq 0, \quad \text{flying time accumulated after } i, \text{ before its check, } k \in \mathcal{C}_H. \quad (13)$$

Variables y_{0ij} and $y_{i\omega j}$ indicate the first and last flight of aircraft j . The hierarchy-reset indicator is not stored as a variable; it is the derived expression

$$q_{ijk} = \sum_{k' \in \mathcal{C}: k' \succeq k} z_{ijk'}. \quad (14)$$

With assignment costs c_{ij} and maintenance costs g_{ijk} , the objective is

$$\min \sum_{i \in \mathcal{F}} \sum_{j \in \mathcal{P}} c_{ij} x_{ij} + \sum_{i \in \mathcal{F}^M} \sum_{j \in \mathcal{P}} \sum_{k \in \mathcal{C}} g_{ijk} z_{ijk}. \quad (15)$$

6 Complete Event-Based Criteria C1–C14

C1: Flight coverage. Every scheduled flight is assigned to exactly one aircraft:

$$\sum_{j \in \mathcal{P}} x_{ij} = 1, \quad \forall i \in \mathcal{F}. \quad (\text{C1})$$

C2: Unique predecessor. Every assigned flight has one predecessor or is the first route flight:

$$\mathbb{1}_{\{\ell \in \mathcal{F}_j^0\}} y_{0\ell j} + \sum_{i: (i, \ell) \in \mathcal{E}} y_{i\ell j} = x_{\ell j}, \quad \forall \ell \in \mathcal{F}, j \in \mathcal{P}. \quad (\text{C2})$$

Source variables $y_{0\ell j}$ are instantiated only for $\ell \in \mathcal{F}_j^0$; the indicator in C2 is notation for this sparse domain.

C3: Unique successor. Every assigned flight has one successor or is the last route flight:

$$y_{i\omega j} + \sum_{\ell: (i, \ell) \in \mathcal{E}} y_{i\ell j} = x_{ij}, \quad \forall i \in \mathcal{F}, j \in \mathcal{P}. \quad (\text{C3})$$

C4: One time-ordered route per aircraft.

$$\begin{aligned} \sum_{i \in \mathcal{F}} y_{0ij} &\leq 1, & \forall j \in \mathcal{P}, \\ \sum_{i \in \mathcal{F}} y_{i\omega j} &= \sum_{i \in \mathcal{F}} y_{0ij}, & \forall j \in \mathcal{P}. \end{aligned} \quad (\text{C4})$$

Airport continuity, turn time, and non-overlap are embedded in $\mathcal{E}$. Since all arcs move forward in time, C2–C4 cannot produce a subtour.

C5: Maintenance requires assignment.

$$z_{ijk} \leq x_{ij}, \quad \forall i \in \mathcal{F}^M, j \in \mathcal{P}, k \in \mathcal{C}. \quad (\text{C5})$$

C6: At most one physical check after a flight.

$$\sum_{k \in \mathcal{C}} z_{ijk} \leq 1, \quad \forall i \in \mathcal{F}^M, j \in \mathcal{P}. \quad (\text{C6})$$

A heavy check satisfies lighter requirements through q_{ijk} , so separate simultaneous A, B, C, and D checks are unnecessary.

C7: Maintenance-airport eligibility. Variables z_{ijk} are instantiated only for $i \in \mathcal{F}^M$. A dense equivalent is

$$z_{ijk} = 0, \quad \forall i \in \mathcal{F} \setminus \mathcal{F}^M, j \in \mathcal{P}, k \in \mathcal{C}. \quad (\text{C7})$$

C8: Maintenance duration and the next flight. An insufficient connection and a maintenance action cannot both be selected:

$$y_{i\ell j} + z_{ijk} \leq 1, \quad \forall (i, \ell) \in \mathcal{E} \setminus \mathcal{E}_k, j \in \mathcal{P}, k \in \mathcal{C}. \quad (\text{C8})$$

For a terminal flight, z_{ijk} is created only if $\sigma_i + \delta_k$ is within the planning horizon. Thus a multi-day check is one fixed-duration virtual leg, not one binary variable per occupied day.

C9: Continuous-time station capacity. Let $b_a(\theta)$ be the available maintenance positions at airport a and time θ . Let Θ contain all possible maintenance start times and all breakpoints at which $b_a(\theta)$ changes. These are the only times at which the left-hand side can increase or the right-hand side can decrease:

$$\sum_{\substack{i \in \mathcal{F}^M : a_i = a \\ k \in \mathcal{C} : \sigma_i \leq \theta < \sigma_i + \delta_k}} \sum_{j \in \mathcal{P}} z_{ijk} \leq b_a(\theta), \quad \forall a \in \mathcal{M}, \theta \in \Theta. \quad (\text{C9})$$

C10: Maintenance hierarchy.

$$q_{ijk} = \sum_{k' \succeq k} z_{ijk'}, \quad \forall i \in \mathcal{F}^M, j \in \mathcal{P}, k \in \mathcal{C}. \quad (\text{C10})$$

C10 is a derived expression and replaces the old daily hierarchy variable.

C11: Flight-hour threshold. For A and B requirements, the flight-indexed counter is active only for an assigned flight and never exceeds the threshold:

$$p_i x_{ij} \leq u_{ijk} \leq T_k x_{ij}, \quad \forall i \in \mathcal{F}, j \in \mathcal{P}, k \in \mathcal{C}_H. \quad (\text{C11})$$

C12: Flight-hour accumulation and reset. For each route connection, let $R_{\ell k} = T_k + p_\ell$ deactivate propagation when the successor arc is not selected. The reset term needs only the state bound T_k :

$$\begin{aligned} u_{\ell j k} &\geq u_{ijk} + p_\ell - R_{\ell k}(1 - y_{i\ell j}) - T_k q_{ijk}, \\ u_{\ell j k} &\leq u_{ijk} + p_\ell + R_{\ell k}(1 - y_{i\ell j}) + T_k q_{ijk}, \\ u_{\ell j k} &\leq p_\ell + T_k(2 - y_{i\ell j} - q_{ijk}), \quad \forall (i, \ell) \in \mathcal{E}, j \in \mathcal{P}, k \in \mathcal{C}_H. \end{aligned} \quad (\text{C12})$$

If $y_{ilj} = 1$ and no qualifying check follows i , the first two inequalities give $u_{\ell jk} = u_{ijk} + p_\ell$. If a qualifying check follows i , the third inequality and C11 give $u_{\ell jk} = p_\ell$. The counter is therefore accumulated and reset on the actual aircraft route, with no day-pair constraints and no endpoint ambiguity.

C13: Pre-horizon accumulated flying time. For a first-flight arc, initialize the counter from known aircraft history:

$$\begin{aligned} u_{ijk} &\geq \Phi_{jk} + p_i - (\Phi_{jk} + p_i)(1 - y_{0ij}), \\ u_{ijk} &\leq \Phi_{jk} + p_i + (\Phi_{jk} + p_i)(1 - y_{0ij}), \quad \forall i \in \mathcal{F}, j \in \mathcal{P}, k \in \mathcal{C}_H. \end{aligned} \quad (\text{C13})$$

C11–C13 replace the old cumulative-flight criterion completely.

C14: Calendar-time spacing without day variables. Let H_0 and H_1 be the horizon boundaries. For $k \in \mathcal{C}_T$, the first deadline is $d_{jk}^0 = H_0 + L_k - \Psi_{jk}$. If $d_{jk}^0 \leq H_1$, require

$$\sum_{i \in \mathcal{F}^M: H_0 \leq \sigma_i \leq d_{jk}^0} q_{ijk} \geq 1, \quad \forall j \in \mathcal{P}, k \in \mathcal{C}_T. \quad (\text{C14a})$$

Every selected check whose next deadline lies in the horizon must be followed by another qualifying check within L_k time units:

$$\sum_{\ell \in \mathcal{F}^M: \sigma_i < \sigma_\ell \leq \sigma_i + L_k} q_{\ell jk} \geq q_{ijk}, \quad \forall i \in \mathcal{F}^M: \sigma_i + L_k \leq H_1, j \in \mathcal{P}, k \in \mathcal{C}_T. \quad (\text{C14b})$$

These timestamp windows require a qualifying check to begin by its due time, matching the application convention used by the day-indexed model. The virtual leg then blocks the aircraft for its complete fixed duration. Thus calendar requirements are preserved without an explicit day set.

7 Obsolete Variables and Constraints

The reformulation removes the explicit day index d , the old variable z_{ijkd} , the daily maintenance variable y_{jkd} , and the daily hierarchy variable γ_{jkd} or η_{jkd} . It also removes one binary per day of a multi-day check and all day-pair accumulation constraints.

The symbol y is retained with a new and precise meaning: y_{ilj} is a sparse successor arc. It provides the aircraft sequence required to propagate the flight-hour counter exactly. The maintenance variable is now only z_{ijk} .

8 Complexity and Memory Reduction

Let $n_f = |\mathcal{F}|$, $n_p = |\mathcal{P}|$, $n_c = |\mathcal{C}|$, n_d be the old number of days, and $n_e = |\mathcal{E}|$. The old maintenance-choice tensor contains $O(n_f n_p n_d n_c)$ binaries. The new tensor contains only $O(n_f n_p n_c)$ binaries and is instantiated sparsely over $\mathcal{F}^M$:

$$O(n_f n_p n_d n_c) \longrightarrow O(n_f n_p n_c). \quad (16)$$

The complete event-based model contains

$$O(n_f n_p + n_e n_p + n_f n_p n_c) \quad (17)$$

binary variables, plus $O(n_f n_p |\mathcal{C}_H|)$ continuous counters. The sparse successor variables add $O(n_e n_p)$, but n_e is normally much smaller than n_f^2 because airport and time incompatibilities eliminate most arcs. Most importantly, the maintenance model no longer scales multiplicatively with the number of planning days.

Table 1: Principal variable counts.

Component	Day-indexed model	Event-based model
Flight assignment	$n_f n_p$	$n_f n_p$
Maintenance choice	$n_f n_p n_d n_c$	$n_f n_p n_c$
Daily maintenance	$n_p n_d n_c$	removed
Daily hierarchy	$n_p n_d n_c$	derived, not stored
Route sequence	implicit	$n_e n_p$ sparse binaries
Hour state	day-pair constraints	$2n_f n_p$ continuous variables

Remark 1. *The day dimension is removed, but calendar information is not. All timestamps must use one absolute, horizon-wide time scale; otherwise overnight gaps and C/D elapsed-time requirements cannot be recovered correctly.*

9 Model Class: LP, QP, or Convex?

With binary assignment/connection/maintenance variables ($x_{ij}, y_{ilj}, z_{ijk} \in \{0, 1\}$), C1–C14 is a **Mixed-Integer Linear Program (MILP)**.

LP. The exact integrated model is not a pure LP because of integrality. However, its LP relaxation (replace $\{0, 1\}$ by $[0, 1]$) is linear and convex, and is useful for bounds and diagnostics.

QP. A quadratic objective is not required for this reformulation. If quadratic terms are added (for example, smoothness penalties), the model becomes MIQP, not simpler than MILP for the core TAP structure.

Convexity. The continuous relaxation is convex (polyhedral). The original integer model is combinatorial and therefore non-convex in decision space.

When can it be pure LP? Only under strong simplifications, such as fixed aircraft routes/flight assignment known in advance, after which maintenance timing may reduce to a continuous feasibility LP. For integrated assignment plus maintenance timing, MILP is the correct class.

10 Implementation Plan for Code and Paper Updates

This plan updates the codebase and thesis artifacts to the event-based maintenance reformulation (no day-indexed maintenance variables).

Phase 1: Freeze baseline and define acceptance tests

1. Freeze a baseline commit and keep current regression tests green.
2. Add acceptance criteria:
 - C1–C4 route feasibility unchanged,
 - maintenance duration blocking via virtual self-loop events,
 - C/D calendar windows validated by absolute timestamps,
 - A/B hour reset validated on route replay,
 - objective decomposition (flight vs maintenance) preserved.

Phase 2: Core model refactor in code

1. Replace day-indexed maintenance tensors by event-linked variables z_{ijk} in the new implementation path.
2. Build sparse connection sets $\mathcal{E}$ and $\mathcal{E}_k$ from timestamps.
3. Encode maintenance as fixed-duration virtual self-loop event after flight i at airport a_i .
4. Implement C5–C10 in event form (assignment link, single physical check, maintenance-station eligibility, duration blocking, hierarchy).

Phase 3: Time-state constraints migration

1. Implement C11–C13 with route-propagated hour-state variable u .
2. Implement C14a/C14b with absolute-time windows for C/D checks.
3. Remove legacy day-pair C13 logic from the active path after parity checks pass.

Phase 4: Validation and parity protocol

1. Unit tests: new tests for maintenance-event placement, C8 blocking, C14 deadline enforcement, and hierarchy reset semantics.
2. Small-instance parity: compare legacy and event models on matched feasible instances; verify same optimal objective where semantics overlap.
3. Independent replay validator: reconstruct route timeline and verify all maintenance limits from generated schedules.
4. Stress runs: scalability checks on $h=7/15/21$ classes with fixed solver/time limits.

Phase 5: Thesis and paper updates

1. Update formal model chapter with C1–C14 event-based equations and variable table (x, y, z, u, q) .
2. Add a short subsection explaining why z_{jk} is insufficient and why event anchor i is required.
3. Regenerate computational tables/figures and refresh captions to state solver settings, instance scope, and parity criteria.
4. Add reproducibility appendix: exact commands, package versions, solver/license notes, and expected outputs.

Phase 6: Packaging and transferability

1. Keep the portable package in `Thesis/code` as the handoff artifact for other devices.
2. Ensure README runbook includes setup, tests, experiment commands, and LaTeX build pipeline.
3. Add one smoke script per platform (Windows/Linux) for quick installation validation.

Exit criteria. Migration is complete when: (i) tests pass, (ii) parity/validation reports are clean, (iii) thesis builds without unresolved references, and (iv) all reproducibility commands in the package README run on a fresh machine.

Proposition 1. *Assume that $\mathcal{E}$ contains exactly the feasible ordinary connections and C8 is imposed for every maintenance-incompatible connection. Every integral solution of C1–C14 then defines time-feasible aircraft routes whose selected maintenance events satisfy duration, station capacity, hierarchy, cumulative flight-hour limits, and calendar-time limits.*

Proof. C1 assigns every flight once. C2–C4 form time-ordered aircraft routes. C5–C8 attach each maintenance event to an assigned flight at an eligible station and reserve its duration. C9 enforces station capacity and C10 applies hierarchy resets. C11–C13 propagate and bound the flight-hour counter along each selected route. C14 links qualifying calendar-time checks from the known pre-horizon state through the planning horizon. Hence all stated operating and maintenance conditions hold. $\square$